

선형회귀

"몇 개?"를 맞추는 문제.

이 상품이 **얼마나 팔릴지**, 집값이 **얼마일지** 같은 숫자요.

Q1 분류와 뭐가 다를까

분류 = "어느 칸에 넣을까"

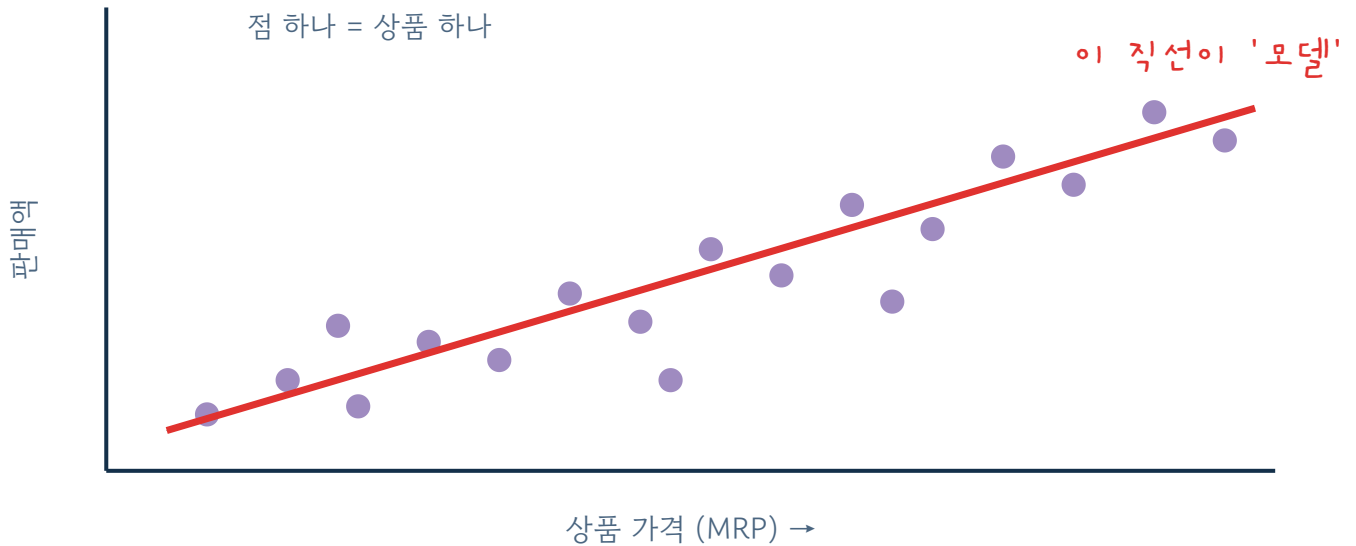
회귀 = "몇 이라고 할까"

	분류	회귀
답	스팸 / 정상, 1등급 / 2등급	3,780원, 27.5도, 2억 4천만원
답의 개수	정해져 있음	무한대 (사이 값이 다 가능)
맞다/틀리다	맞거나 틀리거나	얼마나 가까운지로 잴다

회귀에는 '정답'이 없습니다. 판매액을 3,780원이라고 예측했는데 실제가 3,800원이면, 틀렸지만 꽤 잘한 거죠. 그래서 채점 방식도 완전히 달라집니다.

Q2 직선 하나 긋는 게 전부입니다

슈퍼마켓 데이터로 예를 들어볼게요. **상품 가격(MRP)**으로 **판매액**을 맞춰봅니다.



흩어진 점들 사이로 **가장 잘 어울리는 직선 하나**를 찾는 것. 그게 선형회귀입니다.

직선은 딱 두 개의 숫자로 정해집니다.

$$\text{판매액} = 15.58 \times \text{가격} + 3.47$$

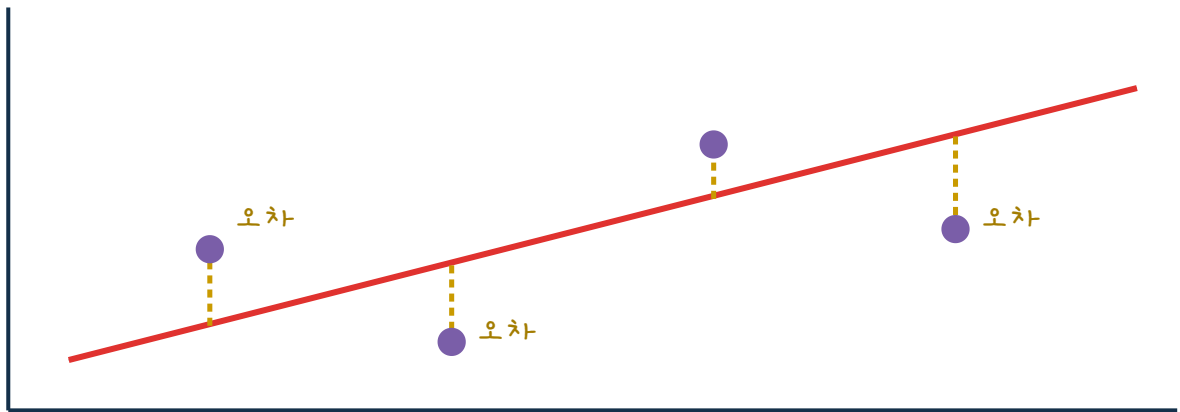
기울기(coef) · 절편(intercept)

이름	뜻
기울기 15.58	가격이 1 오르면 판매액이 15.58 오른다
절편 3.47	가격이 0일 때의 출발점 (그래프가 세로축과 만나는 곳)

"학습한다"는 건 결국 **이 두 숫자를 찾는 것**입니다. 그게 전부예요.

Q3 '가장 잘 어울리는' 직선이란

직선은 무한히 그을 수 있는데, 컴퓨터는 어떤 게 제일 좋은 직선인지 어떻게 알까요?



노란 선(실제와 예측의 차이)을 전부 더했을 때 가장 작아지는 직선이 정답

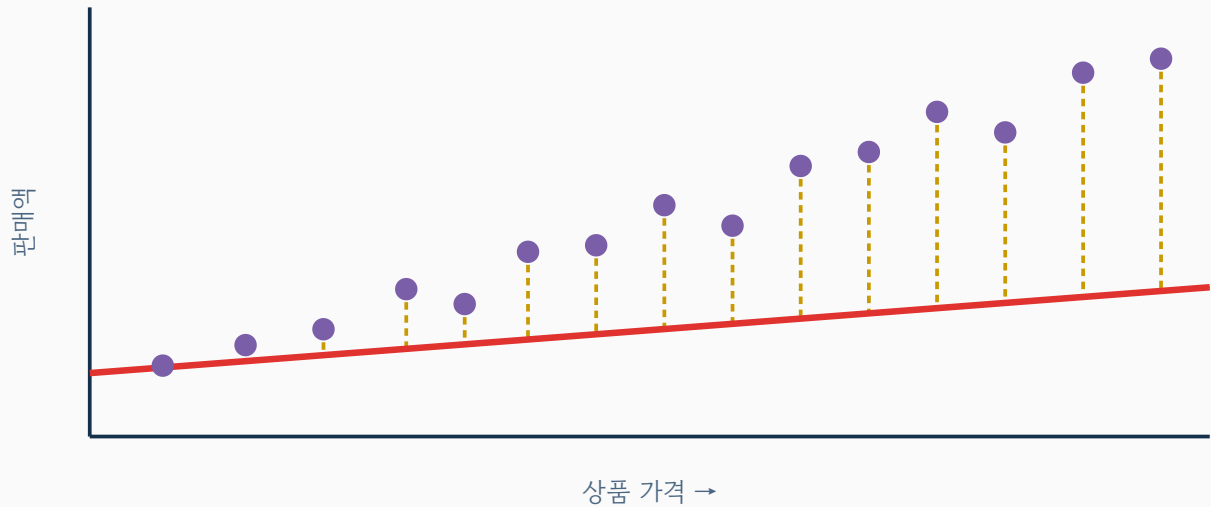
점과 직선 사이의 세로 거리를 **오차(잔차)**라고 부릅니다.

오차는 그냥 더하면 안 됩니다. 위로 +5, 아래로 -5면 합이 0이 돼서 "완벽하다"는 착각이 생기거든요. 그래서 **제곱해서 더합니다**. 제곱하면 음수가 사라지고, **크게 틀린 것에 더 큰 벌점이 붙어요**.

Q4 직접 직선을 그어보기

가장 좋은 직선 찾기

기울기와 높이를 움직여서 오차를 최대한 줄여보세요. 컴퓨터가 찾은 답과 비교할 수 있습니다.



기울기 (가격 1당 판매액 증가)

4.0

절편 (출발 높이)

600

오차 합계 (작을수록 좋음)

1433

컴퓨터의 답 보기

직선이 점들과 한참 떨어져 있습니다.

이 노가다를 컴퓨터가 순식간에, 그리고 정확하게 해주는 게 `LinearRegression().fit()` 한 줄입니다.

Q5 채점표 — MAE, RMSE, R²

회귀는 맞다/틀리다가 없으니, "평균적으로 얼마나 빗나갔나"로 채점합니다.

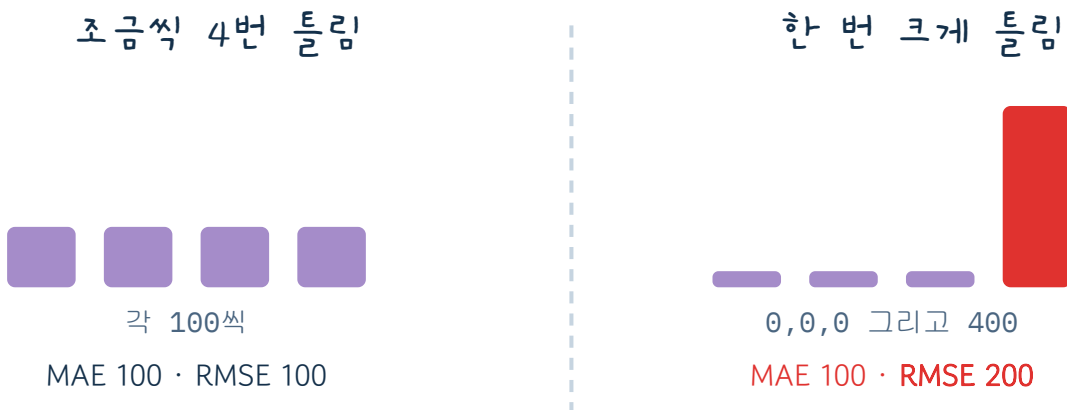
① MAE — 평균 몇 원 틀렸나

빗나간 거리를 그냥 평균 낸 값입니다. 가장 이해하기 쉬운 점수예요.

MAE 800 = "평균적으로 800원씩 빗나갔다"

② RMSE — 크게 틀린 걸 더 미워하는 점수

제공했다가 다시 뿌리를 씩은 값. 크게 빗나간 예측에 벌점을 크게 매깁니다.



MAE는 둘 다 100이지만 RMSE는 오른쪽을 두 배로 나쁘게 봅니다. 대형 사고를 잡아내는 점수예요.

③ R² — 이 모델이 쓸모가 있긴 한가

0~1 사이의 값입니다. 시험 성적처럼 읽으면 됩니다.

R ²	뜻
1.0	완벽. 점이 전부 직선 위에 있음
0.57	판매액이 왜 달라지는지 중 57%를 설명함 (예제 결과)
0	그냥 전부 평균값으로 찍는 것과 똑같음 → 모델 무의미
음수	평균으로 찍는 것보다 더 못함. 원가 잘못됨

MAE·RMSE는 단위가 있는 점수(원, 도, 개)라서 "이 정도면 쓸 만한가?"를 판단하기 어렵습니다. 800원 오차는 3,000원짜리 상품엔 크지만 300만원짜리엔 작죠.

R²는 단위가 없어서 어느 문제에서든 비교할 수 있습니다. 그래서 셋을 같이 봅니다.

Q6 특성을 늘리면 — 다중 선형회귀

가격 하나만으로 판매액을 맞히기엔 부족합니다. **매장 종류, 매장 크기, 상품 무게**도 같이 보면 어떨까요?

$$\text{판매액} = 15.6 \times \text{가격} + 220 \times \text{매장크기} - 90 \times \text{매장종류} + \dots + 3.47$$

특성이 늘어도 **원리는 똑같습니다**. 곱해서 더할 뿐이에요. 다만 그림으로는 더 이상 직선이 아니라 눈에 안 보이는 평면이 됩니다.

여기서 문제가 하나 생깁니다. **매장 종류는 숫자가 아니라 글자**라는 것.

글자 그대로면

매장종류
대형마트
편의점
창고형

→
원핫 인코딩

대형	편의점	창고형
1	0	0
0	1	0
0	0	1

항목마다 칸을 만들고, 해당하면 1 아니면 0

이걸 **원핫 인코딩**이라고 합니다. 판다스에서 `pd.get_dummies()` 한 줄이면 끝나요.

왜 그냥 대형마트=0, 편의점=1, 창고형=2 로 안 할까요?

그러면 모델이 "**창고형이 편의점보다 2배 크다**"고 오해합니다. 순서도 크기도 없는 이름인데 말이죠. 그래서 칸을 따로 나눠줍니다.

Q7 코드로 보기

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# ㉠ 맞힐 것(y)과 재료(X) 나누기
y = train['Item_Outlet_Sales']
X = train.drop(columns=['Item_Outlet_Sales'])

# ㉡ 글자 컬럼을 0/1로 (train과 test를 합쳐서 해야 칸이 어긋나지 않는다)
all_oh = pd.get_dummies(pd.concat([X, X_test], axis=0))
X = all_oh.iloc[:len(train)]
X_test = all_oh.iloc[len(train):]

# ㉢ 분할 - 분류와 달리 stratify가 없다 (나눌 등급 자체가 없으니까)
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2, random_state=0)

# ㉣ 학습 = 기울기와 절편 찾기
lr = LinearRegression().fit(X_tr, y_tr)
print('기울기:', lr.coef_, '| 절편:', lr.intercept_)
pred = lr.predict(X_val)

# ㉤ 채점
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
print('MAE :', mean_absolute_error(y_val, pred)) # 약 800
print('RMSE:', np.sqrt(mean_squared_error(y_val, pred))) # 약 1060
print('R2 :', r2_score(y_val, pred)) # 약 0.57
```

분류에서 쓰던 `predict_proba`, `f1_score`, `stratify` 는 회귀에선 전부 안 씁니다. 확률도 없고 등급도 없으니까요.

Q8 직선으로 부족하면

R^2 0.57은 절반을 조금 넘는 수준입니다. 나머지 43%는 설명을 못 하고 있다는 뜻이죠. 세상 일이 직선처럼 딱 떨어지지 않아서 그렇습니다.

이럴 때는 휘어질 수 있는 모델로 바꿉니다. 같은 데이터, 같은 채점표를 그대로 쓰면서 모델만 갈아끼우면 돼요.

```
from lightgbm import LGBMRegressor

model = LGBMRegressor(n_estimators=500, learning_rate=0.05)
model.fit(X_tr, y_tr)
pred = model.predict(X_val) # 채점 코드는 그대로 재사용
```

이름 끝이 `Regressor` 면 숫자를 맞추는 모델, `Classifier` 면 칸을 고르는 모델입니다. 이 두 글자만 보면 됩니다.

Q9 한 장 정리

	분류	회귀
맞히는 것	칸(라벨)	숫자
모델 이름	~Classifier	~Regressor
채점	정확도, f1, ROC-AUC	MAE, RMSE, R ²
분할	stratify=y 사용	사용 안 함
확률	predict_proba 있음	없음

이것만 기억하면 됩니다

- ① 학습이란 결국 기울기와 절편을 찾는 일이다
- ② 좋은 직선 = 오차의 제곱을 다 더했을 때 가장 작은 직선
- ③ MAE는 얼마나 빗나갔나, R²는 쓸모가 있나 — 둘 다 봐야 한다

Q10 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 당뇨 진행도 숫자 예측

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X, y = load_diabetes(return_X_y=True)      # 1년 뒤 당뇨 진행도(숫자 예측)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
print('R2:', round(r2_score(y_te, model.predict(X_te)), 3))
```

`load_diabetes(return_X_y=True)`

1년 뒤 당뇨 진행도(연속된 숫자)를 예측하는 회귀 데이터.

`LinearRegression().fit`

가장 잘 맞는 직선(면)을 찾습니다.

`r2_score(...)`

R2(1이면 완벽, 0이면 평균 수준)로 평가.

LEVEL 2 복잡 — 캘리포니아 집값 + MAE/RMSE/R2

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = fetch_california_housing(return_X_y=True) # 캘리포니아 집값(특성 8개)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
pred = model.predict(X_te)
print('MAE :', round(mean_absolute_error(y_te, pred), 3))
print('RMSE:', round(np.sqrt(mean_squared_error(y_te, pred)), 3))
print('R2  :', round(r2_score(y_te, pred), 3))
```

fetch_california_housing

캘리포니아 집값 데이터(특성 8개, 처음엔 인터넷 다운로드).

mean_absolute_error / RMSE

평균 얼마나 틀리나(MAE), 큰 오차에 민감한 RMSE.

r2_score

세 지표를 함께 봐서 회귀 성능을 판단.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

url = 'https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/insurance.csv'
df = pd.read_csv(url)
X = pd.get_dummies(df.drop(columns='charges')) # 글자(sex/smoker/region) -> 0/1
y = df['charges'] # 의료비(숫자 예측)

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
print('R2:', round(r2_score(y_te, model.predict(X_te)), 3))
```

```
pd.read_csv(url)
```

캐글 보험(의료비) 데이터를 URL에서 바로.

```
pd.get_dummies(df.drop(columns='charges'))
```

정답(charges)만 빼고, 글자 열은 0/1로.

```
LinearRegression -> r2_score
```

의료비(숫자)를 예측하고 R2로 평가.